ASSIGNMENT 4:

SPARK-BASED CLUSTERING, INVERTED INDEX, AND PAGERANK SYSTEM

B Mugundhan

M25DE1037

m25de1037@iitj.ac.in

GitHub Repo Link:

https://github.com/Mugundh97B/Spark-Clustering-PageRank-System.git

Introduction

This assignment focuses on implementing three fundamental large-scale data processing techniques using PySpark:

1. **Clustering Algorithms** (k-center, k-means++)
2. **Inverted Index and TF-IDF based Search System**
3. **PageRank Algorithm on Graph Data**

The objective is to understand how distributed computation can be used to efficiently process large datasets and implement scalable algorithms.

Part 1: Clustering

Problem Understanding

Clustering aims to group similar data points together. In this assignment:

- Dataset: **UCI SpamBase**
- Size: **4601 emails × 58 features**
- Goal: Compare three methods:
    - k-center (Farthest First Traversal)
    - k-means++
    - Coreset (k-center → k-means++)

Algorithms Used

1. k-center (Farthest First Traversal)

- Selects points that are farthest from existing centers
- Greedy approach
- Does not directly optimize k-means objective

2. k-means++

- Probabilistic initialization
- Chooses centers based on squared distance
- Provides better clustering quality

3. Coreset Approach

- First reduce dataset using k-center (k1 > k)
- Then apply k-means++ on reduced dataset
- Improves efficiency with slight accuracy trade-off

Results

Method	kmeansObj
kcenter	~206,587
kmeans++	~77,727
Coreset (k1=10)	~156,735
Coreset (k1=20)	~140,166

Observations

- kmeans++ achieves the lowest objective value, indicating better clustering quality.
- The kcenter algorithm performs worst, as expected due to its greedy nature.
- The coreset strategy improves over plain kcenter but does not outperform kmeans++.
- Increasing coreset size (k1 from 10 → 20) improves performance significantly.

This demonstrates a **trade-off between computational efficiency and clustering quality**.

Assumptions

- First data point is chosen as initial center in k-center
- Random seed fixed for reproducibility in k-means++
- Squared Euclidean distance used

Part 2: Inverted Index and Search Engine

Problem Understanding

An inverted index maps:

word → list of (page, positions)

This enables fast text search similar to search engines.

Features Implemented

- Stop word removal
- Case normalization
- Punctuation handling
- Singular normalization (e.g., stacks → stack)
- Position tracking (1-based indexing)
- Query processing

Output Result

Result: ALL 11 queries CORRECT

Observations

- The system correctly retrieves:
    - Pages containing a word
    - Positions of words in pages
- Sorting and formatting match expected outputs exactly.

TF-IDF Demonstration

Result:

- Word: **"stack"**
- Appears in all documents

Observation:

- IDF = log(N / n_w) = log(1) = 0
- TF-IDF = 0 for all pages

This shows that **very common words are not useful for ranking**, which is a key idea in TF-IDF.

Assumptions

- Stop words still count toward position indexing
- "+" symbol is preserved (for terms like C++)
- Pages sorted alphabetically

Part 3: PageRank (Spark)

Problem Understanding

PageRank assigns importance to nodes in a graph based on incoming links.

Formula:

$r=(1-\beta)/n+\beta Mr$

Implementation Details

- Spark RDD-based computation
- Edge deduplication
- Broadcast variables for efficiency
- Iterative updates (40 iterations)

Small Graph Result

Top score ≈ 0.0357 (Expected ≈ 0.036)

Matches expected output

Whole Graph Result

Top 5 Nodes

Rank	Node	Score
1	263	0.002020
2	537	0.001943
3	965	0.001925
4	243	0.001852
5	285	0.001827

Bottom 5 Nodes

Rank	Node	Score
1	558	0.000328
2	93	0.000351
3	62	0.000353
4	424	0.000354
5	408	0.000387

Validation

PageRank sum ≈ 1.000000

Convergence Analysis

- Rapid increase in initial iterations
- Stabilization after ~5 iterations
- No significant change after convergence

Observations

- PageRank converges quickly
- 40 iterations are sufficient
- High-ranked nodes receive more incoming links
- Low-ranked nodes have fewer or weaker connections

Assumptions

- No dangling nodes (given dataset property)
- Graph edges are deduplicated
- Node IDs are not assumed sequential

Conclusion

This assignment demonstrates how large-scale data processing can be efficiently implemented using Spark.

Key Takeaways:

- k-means++ provides the best clustering quality
- Coreset methods balance efficiency and accuracy
- Inverted index enables efficient text search
- TF-IDF down-weights common words
- PageRank effectively ranks nodes in a graph
- Spark enables scalable computation for all tasks

Final Summary

Part	Outcome
Clustering	kmeans++ best, coreset improves efficiency
Search Engine	All queries correct
TF-IDF	Common words get zero importance
PageRank	Correct ranking and convergence